

Testing Documentation

Unit Testing:

1-Game Logic:

1.1-switchPlayer:

Checked for the functionality of swapping the player's symbol where it always starts with X in a TicTacToe game then swaps to O and so on.

1.2-checkWin:

Filled only 1 row/column/diagonal on the board at a time but also while checking for all possible win conditions then checked if the function detects a draw condition by emptying the board and in the end checked if the function detects a not winning condition if a symbol winning is short by 1 symbol.

1.3-checkdraw:

Put a couple of win conditions on the board then checked if checkdraw would recognize them as "not draw" conditions then I filled the board with a couple of symbols that wouldn't lead to a win but a draw and checked if the function recognized it as a draw or not.

1.4-makemove:

Checked if the function takes only valid inputs for rows and columns (from zero to two) then checked for the placement of the move on the right column on the right row then in the end checked that the function doesn't allow a move on a place that already had a symbol inside.

2-AI:

2.1-isMovesLeft:

Checked if it correctly returns a value according to the board's state, one time the board wasn't filled and the function got tested if it detects the unfilled board correctly or not and one time the board got filled and the function got tested if it recognizes the board as filled or not.

2.2-Easy and medium AI:

Ran 100 games on the test between medium and easy difficulty and due to medium's implementation it might not pick the best move for the whole game so it is nondeterministic but it should win more than 50% of the games as bare minimum indicating that medium is higher in the difficulty hierarchy than easy.

2.3-Hard and medium AI

Ran 100 games between hard and medium AI and due to medium AI's implementation it might pick the best move for the whole game so the hard should win more than 50% of the games indicating that hard is higher in the difficulty hierarchy than medium.

Since hard beat medium that beat easy then → Hard>Medium>Easy in difficulty

2.4-Hard Vs. Hard AI:

To check if the hard AI implements a perfect game I ran 100 test games between the 2 difficulties each game must end in a draw since a perfect game against a perfect game will always result in a draw

3-Database:

3.1-Database correct allocation:

Created a user with a specific username and password then checked if the user was actually allocated through the validateUser function, then checked if the validateUser function doesn't recognize an entry with an invalid username but a valid password and in another case with an invalid password and a valid username and lastly in the case of both being invalid the username and the password

3.2-delete User:

Created a user then checked if the delete function doesn't delete an entry in the database if it doesn't exist in the first place, then made it delete a valid entry then checked if the entry was deleted or not

3.3-game history retrieval:

Created 8 game data sets in the database knowing who both players were, who the winner was, and the board's state then retrieved them back testing if both players were stored correctly, the winner was stored correctly, and the board's state was stored correctly

4-Overwrite game:

4.1-remembering the last move:

Overwrite game uses the same logic implemented in the classic game but with its extra feature, tested if it doesn't allow overwrite moves in the case that the previous move was for the opponent for both X and O and also tested if it allows an overwrite move in the case that it wasn't the opponent's last move